AIQA — Fresh Full-Project Evidence-Based Audit & Implementation Verification

Audit date: 24 August 2026 **Repository:** `D:\AIQA\.claude\worktrees\run-this-9eeb2e` (git worktree, branch `claude/run-this-9eeb2e`, working tree clean at audit start, HEAD `0ab75de`) **Auditor posture:** independent, evidence-only. No claim in any prior `AIQA_*_REPORT.md` file (there are ~90 of them in the repo root) was used as evidence. Every classification below is backed by a current file:line citation, a passing/failing test, a live runtime observation made in this session, or an explicit statement that evidence was insufficient.

---

1. Executive Summary

AIQA is a two-tier system: a genuinely substantial Rust/Axum orchestrator backend (`services/orchestrator/`, ~15,450 lines across 22 source files) with real SQLite persistence, real bearer-key auth with three enforced roles, real project-level multi-tenant isolation, a real policy/approval-gate engine, a real evidence-capture pipeline, five real per-tool test executors (Playwright, Newman, Pytest, Schemathesis, Maestro), and a real central AI provider/model router with genuine Ollama integration, capability-based scoring, AUTO/MANUAL routing, and tested fallback behavior — plus a React/Vite frontend (`forentend/`, 70+ source files) that is a real, well-tested consumer of most of that backend (Dashboard, Reports, Evidence Viewer, Audit Trail, Telemetry, and the AI Settings panel are all genuinely wired end to end).

The single most consequential finding of this audit: the product's headline capability — AI-powered test case generation — does not work as shipped, despite a complete, well-tested, real implementation of it existing in the backend. `TestCaseGeneratorPage.tsx`'s "Generate Test Cases" button never calls the backend at all; it runs a 100%-client-side, hardcoded, per-category template generator. `selectedSkillIds` is captured in UI state and then never transmitted. Custom skills are explicitly session-only. Document upload never reads a file's content. This is honestly disclosed in-app (a visible "LOCAL TEMPLATE GENERATION" badge, and a literal `aiAnalysis` string admitting "no AI model was called") — so it is not a fabrication in the deceptive sense — but it means AIQA's central value proposition is currently inert on the only path a real user would exercise.

Two other systemic patterns matter:

- **What's real is unusually well-tested.** 206/206 backend tests pass (`cargo test`, 2 ignored — live-Ollama integration tests). 371/371 frontend tests pass (`vitest run`, 20 files). These are not shallow "renders without crashing" tests — this audit independently re-ran the backend suite live and confirmed named tests asserting cross-tenant isolation, RBAC denial, truthful fallback labeling, and SSRF-guard behavior against real cloud-metadata IP ranges.
- **What's fake is, with a few exceptions, honestly disclosed.** The `SimulatedExecutor`, the local-template generator, and every "not implemented" stub page carry in-app or in-code disclosure. The exceptions — a fabricated "Connected" network-status check, three pre-seeded fake "generated" test cases shown with no disclosure on first load, and a Settings page that silently fake-persists most of its tabs behind a real-looking "Configuration Saved" toast — are flagged below as the audit's most serious **Fake/Stub/Placeholder** findings.

Verdict: CONDITIONAL GO. See §44.

---

2. Audit Scope

All 46 areas listed in the audit brief were covered: frontend, backend, API contracts, persistence, authentication, RBAC, multi-tenancy, test case generation, AI-powered generation, skills, custom skills, MCP, AI provider/model discovery, health checks, capability matching, routing, AUTO/MANUAL modes, AI settings, fallback, test execution, the 17 named testing disciplines, reporting, evidence, audit logging, metrics, error handling, security, fake/stub/placeholder content, test coverage/quality, and production readiness.

Out of scope by absence, not by exclusion: `apps/web/` was inspected only enough to confirm its status (a separate, buildable, likely-abandoned duplicate frontend — see §6.5); `services/workers/` was inspected only enough to confirm it is an unwired Python-adapter contract stub for a future distributed-worker architecture (README's own claim, verified true).

---

3. Audit Methodology

1. **Parallel, independent code research.** Six read-only research passes were run concurrently, each scoped to a different subsystem, each instructed to distrust prior report files and cite file:line evidence for every claim, and each instructed to explicitly mark `UNVERIFIED` rather than guess:

- AI infrastructure, provider/model discovery, routing, AUTO/MANUAL, fallback, AI Settings panel
- Test Case Generator, Skills, Custom Skills, MCP
- Execution Hub, the five real executors, SimulatedExecutor, the 17 named testing disciplines
- Authentication, RBAC, multi-tenancy, persistence, policy/approval gates, audit log
- Dashboard, Reports, Evidence Viewer, Audit Trail, Telemetry, API contract drift, error handling
- Repo-wide fake/stub/placeholder sweep, security audit, dependency audit, code-quality note

2. **Direct command execution** (not delegated, so exact pass/fail counts are first-hand): `git status`/`log`, `cargo test`/`cargo build` (orchestrator), `npm run lint` (`tsc --noEmit`)/`npm test` (`vitest run`)/`npm run build` (`forentend` and, for comparison, `apps/web`), `npm audit --json`.

3. **Live runtime verification.** The orchestrator was actually built and run (`cargo run -p aiqa-orchestrator`, real bootstrap admin key minted and printed once), the frontend dev server was actually built and run (`npm run dev`, Vite on :3000, proxying to :8080), and the two were driven together through a real browser automation pass: health check, API-key configuration through the real Settings UI, a real authenticated `GET /api/v1/projects` request through the Vite proxy, all confirmed via network-request inspection, not assumed.

4. **Cross-verification.** Where two research passes touched the same code (e.g., the AI router surfaced by both the AI-infrastructure pass and the fake/stub sweep), findings were checked for mutual consistency; no contradictions were found.

---

4. Repository Inventory

Evidence-backed, actual-file-listing map (no component below is inferred from a name or a report claim):

```
AIQA/  (git worktree root)
|--- forentend/                        Real, currently-served frontend — React 19 + Vite 6, port 3000
|   |--- src/components/pages/         14 route-level pages (routeConfig.tsx)
|   |--- src/components/dashboard/     AIQualityCoreOrb, ProjectStatusDonut, ProjectExecutionGraph
|   |--- src/components/test-generator/  DocumentUpload, ApplicationSource, GenerationConfig,
|   |                                    GeneratedTestCasesView, SkillsAndMcpPanel, TraceabilityGraph,
|   |                                    SkillManagerModal, McpConnectModal
|   |--- src/context/                  AIQAContext, AuthContext, ProjectContext, RealtimeContext
|   |--- src/hooks/useSkillSelection.ts
|   |--- src/services/aiqaApi.ts, skillRegistry.ts
|   |--- src/types/contract.ts, skills.ts, types.ts
|   +--- 12 *.test.tsx/.test.ts files (vitest)
|
|--- apps/web/                         Separate, smaller (2,299 vs 2,967 modules), buildable duplicate
|                                      frontend — likely abandoned reference app (see §6.5); contains
|                                      a committed real-looking Firebase API key (see §30, AUD-019)
|
|--- services/orchestrator/            Rust/Axum backend — 15,453 lines, 22 .rs files
|   |--- src/main.rs                   entrypoint: SQLite connect+migrate, bootstrap admin key,
|   |                                  crash recovery, executor registry, AI infra bg discovery,
|   |                                  explicit-allowlist CORS, bind 0.0.0.0:8080
|   |--- src/api.rs (2,387 lines)      ~50 routes, auth middleware layer, RBAC checks per handler
|   |--- src/auth.rs (165)             bearer-key hashing (SHA-256), 3 roles, project scoping
|   |--- src/store.rs (1,868)          SQLite (sqlx), WAL mode, all persistence
|   |--- src/executor.rs (879)         5 real tool adapters + SimulatedExecutor, sanitize_token,
|   |                                  SSRF guard, Win32 Job Object process containment
|   |--- src/ai_router.rs (1,107)      provider/model discovery, capability scoring, AUTO/MANUAL,
|   |                                  fallback, real Ollama HTTP client
|   |--- src/ai_generation.rs (662)    skill-to-prompt resolution, structured-output validation,
|   |                                  dedup, enforced traceability
|   |--- src/intelligence.rs (668)     rule-based failure clustering/bug-drafting + opt-in real-LLM
|   |                                  hypothesis augmentation
|   |--- src/engine.rs (730)           run/task execution loop, timeout, cancellation, evidence write
|   |--- src/scheduler.rs (347)        dependency-aware DAG scheduling
|   |--- src/policy.rs (156)           approval-gate decision engine
|   |--- src/evidence.rs (75)          local-disk evidence store, path-traversal guard
|   |--- src/metrics.rs (67)           real Prometheus-format `/metrics`
|   |--- src/webhook.rs (55), state.rs (70), errors.rs (206), domain.rs (441)
|   |--- src/*_tests.rs                api_tests.rs (3,152!), engine_tests.rs (1,480), store_tests.rs
|   |                                  (390), ws_tests.rs (388) — 206 tests total, all passing
|   +--- migrations/ (7 files)         0001 core_execution, 0002 quality_graph_and_intelligence,
|                                      0003 auth_governance_and_audit, 0004 projects,
|                                      0005 test_cases, 0006 ai_settings, 0007 ai_bug_hypothesis
|
|--- executors/                        5 real, runnable per-tool fixture projects
|   |--- playwright/  package.json + playwright.config.ts + tests/sample.spec.ts (real, 3 tests)
|   |--- newman/       package.json + collection.json (real Postman v2.1 collection)
|   |--- pytest/       requirements.txt + test_health.py (real httpx-based tests)
|   |--- schemathesis/ requirements.txt + openapi.yaml + README documenting a real found issue
|   +--- maestro/      flow.yaml (real Maestro flow syntax) + README (documents a real CLI-name
|                      collision with an unrelated tool on this dev machine)
|
|--- services/workers/                  Python adapter *contract stub* only — not wired into the
|                                       orchestrator (confirmed: orchestrator shells out directly
|                                       to executors/, no Redis/worker transport exists)
|
|--- infra/                             docker-compose.yml (Postgres+pgvector, Redis, MinIO) — not
|                                       running, not wired in; migrations/ (SQL mirror, superseded
|                                       by services/orchestrator/migrations/); data/ (the real,
|                                       live SQLite file + evidence blobs actually used)
|
|--- .agents/skills/caveman/           unrelated stray skill artifact, not part of the product
|--- (no .github/ directory)           NOT CONFIGURED — no CI/CD pipeline exists
|--- (no Dockerfile anywhere)          NOT IMPLEMENTED — no containerization
|--- start.bat / stop.bat              real local dual-process launcher (fixed this session's
```

```
|                                   predecessor run to point at forentend, not apps/web)
+--- ~90 AIQA_*_REPORT.md files     historical reports — explicitly NOT used as evidence here
```

---

5. Current Architecture

5.1 Overall System Architecture (arrows verified real)

```
Browser (forentend, Vite :3000, React 19)
   |  Bearer <api key> from localStorage; /api proxied to :8080
   v
Axum HTTP/WS router (services/orchestrator, :8080)
   |  require_auth (SHA-256 key hash lookup) -> RBAC require_role -> project-scope check
   v
+-------------------------------------------------------------------------+
|  api.rs handlers (~50 routes)                                           |
|    |-- projects / API-keys / audit-log  ------> store.rs -> SQLite      |
|    |-- runs/tasks (create/approve/reject/cancel)                        |
|    |      +--> policy.rs (approval gate) -> engine.rs -> scheduler.rs   |
|    |              +--> executor.rs -> real subprocess (5 tools)         |
|    |                     +--> evidence.rs -> local disk + store.rs      |
|    |-- test-cases (CRUD + generate-ai)  ------> ai_generation.rs        |
|    |                                          +--> ai_router.rs         |
|    |                                                 +--> Ollama HTTP   |
|    |-- findings / bug-drafts  ----------------> intelligence.rs         |
|    |-- dashboard / kpis / metrics  ------------> store.rs SQL aggregates|
|    +-- ai/status /settings /refresh  ---------> ai_router.rs            |
+-------------------------------------------------------------------------+
```

Verified real at every arrow above via live `cargo test` execution and/or direct code reading (agent citations in §7–§29). **Not real / not reached** in the live product: the `generate-ai` arrow exists and works in isolation, but no frontend code calls it (§13).

5.2 AI Generation Flow — as implemented in the backend vs. as actually reachable from the UI

```
Implemented in ai_generation.rs + ai_router.rs (real, tested, UNREACHABLE from the shipped UI):
  Requirement text + Skills
      |
      v
  Skill Resolver (builtin_instruction / custom description)
      |
      v
  Prompt Builder (one distinct instruction block per selected skill)
      |
      v
  AI Model Router (route_among: score, select, AUTO or MANUAL)
      |
      v
  Provider Executor (Ollama: real POST /api/generate — the only real one)
      |
      v
  Structured Output Validation (serde schema; malformed -> Failed, never faked)
      |
      v
  Enforced Traceability (source_reference re-checked as literal substring of real text)
      |
      v
  Deduplication (normalized-title exact match)
      |
      v
  GenerateAiTestCasesResponse { test_cases, generation_method: "ai"|"fallback", ... }

Actually reachable from TestCaseGeneratorPage.tsx (what a real user sees):
  Selected Categories (NOT Skills)
      |
      v
  switch(category) { hardcoded per-category template string }   <- setTimeout, 100% local
      |
      v
  Generated Test Cases (labeled "LOCAL TEMPLATE GENERATION", honest aiAnalysis disclaimer)
```

5.3 Model Routing Flow (real, backend-enforced, verified by outcome-based tests)

```
AiTask (e.g. TestCaseGeneration)
   |
   v
required_capabilities(task)  — fixed, documented per-task capability list
   |
   v
Discovered providers × models (Ollama: real /api/tags enumeration;
                               Gemini/OpenAI/Anthropic: env-var-presence only, 0 models ever)
   |
   v
Health (Ollama: real HTTP round-trip, TTL-cached 60s; cloud: never "Healthy", only "Available"/"NotConfigured")
   |
   v
score_model()  — +100/+40/+5 per required capability at Strong/Basic/Unknown,
                 disqualifying on RequiredCapability::None, provider-health bonus
   |
   v
RoutingMode: AUTO -> best score wins  |  MANUAL -> exact preferred provider/model or explicit error
   |
   v
Fallback (AUTO + fallback_enabled only): re-score excluding failed candidate, retry once
   |
   v
GenerationOutcome { Success | Failed } — never silently faked as Success
```

---

6. Frontend Audit

Status: VERIFIED (build/test health), PARTIAL (product completeness)

- `npx tsc --noEmit` (via `npm run lint`): **PASS, 0 errors** (after excluding one transient Vite temp-file artifact that doesn't exist once the dev server settles — confirmed reproducible-then-gone, not a real type error).
- `npx vitest run`: **PASS — 20 test files, 371 tests, 0 failures**, 134.9s.
- `npx vite build`: **PASS** — 2,967 modules transformed, `dist/` produced. One real quality warning: main JS chunk is 1,072.16 kB (303 kB gzipped) — no code-splitting configured (see AUD-025).
- Routing (`routeConfig.tsx`): 14 routes, each carrying an explicit `designStatus` field — a real, honest self-declaration of implementation maturity that this audit independently confirmed matches actual page behavior in every case checked (`unsupported-not-implemented` pages really are stubs; implemented pages really do call real endpoints).
- Real, verified-wired pages: Dashboard, Reports, Evidence Viewer, Audit Trail, Telemetry, Settings (AI Models tab only), Execution Hub.
- Honest stub pages (self-disclosed `EmptyState`, no fabricated data): RBAC Testing, Mobile Testing, API Testing, Custom Testing, Feature Map/Flow, AIQA Orchestrator (legacy), Test Case Validator (redirects to Generator).
- **Product-critical gap**: Test Case Generator page (§13).
- **Disclosed-but-misleading gap**: Settings page's non-AI tabs (§31, AUD-007).

---

7. Backend Audit

Status: VERIFIED — high maturity

`cargo test` (live run, this session): **206 passed; 0 failed; 2 ignored** (the 2 ignored tests require a live Ollama server — `#[ignore]`-gated, not silently skipped, correctly excluded from the default run). `cargo build`: succeeds (debug profile, 1m54s cold). No compiler warnings observed in the captured build log.

Module-by-module real-vs-stub status (full detail in later sections):

Module	Real?	Evidence
`auth.rs`	VERIFIED	SHA-256 hashed keys, 3 roles, project scoping, audited failures
`store.rs`	VERIFIED	Real SQLite, WAL, parameterized queries throughout, restart-recoverable
`executor.rs`	VERIFIED	5 real subprocess adapters, sanitize_token, SSRF guard, Job Object containment
`engine.rs` / `scheduler.rs`	VERIFIED	Real DAG scheduling, timeout, cancellation, crash recovery
`policy.rs`	VERIFIED	Real approval-gate decision, teeth confirmed (Deny never spawns)
`evidence.rs`	VERIFIED	Real file capture + path-traversal guard
`ai_router.rs`	PARTIAL	Real for Ollama; Gemini/OpenAI/Anthropic are env-presence stubs
`ai_generation.rs`	VERIFIED (unreached)	Real, tested; no frontend caller
`intelligence.rs`	PARTIAL	Auto pipeline rule-based (as documented); opt-in real-LLM hypothesis exists, also unreached by frontend
`metrics.rs`	VERIFIED	Real Prometheus text from real SQL COUNTs

---

8. API Contract Audit

Status: PARTIAL — one major, load-bearing gap

Six major shared types (`DashboardResponse`, `KpisResponse`, `AuditRow`, `TestIntelRow`, `RunEvent`, `EvidenceRow`, `ApiErrorEnvelope`) were diffed field-by-field between `forentend/src/types/contract.ts` and the actual Rust `#[derive(Serialize)]` structs: **no drift found** in any of them.

The one confirmed gap is severe: `POST /api/v1/projects/:project_id/test-cases/generate-ai` (`api.rs:756`) is a real, complete, RBAC-gated, project-scoped endpoint accepting `{ requirement_text, skills: Vec<SkillInputBody> }` — and `forentend/src/services/aiqaApi.ts` has **no client function for it at all** (grep for `generate-ai`/`generateAi` across all of `forentend/src`: zero matches). The same is true of the newer `POST .../findings/:finding_id/ai-hypothesis` endpoint. Both are real, backend-only capabilities with zero frontend transport.

Error-handling contract: **VERIFIED**. Every handler returns `{error:{code,message,details,request_id}}` (`errors.rs`); a missing/invalid key returns `401 UNAUTHENTICATED` (confirmed live this session via `curl`, and confirmed in source at `auth.rs`/`errors.rs`); the frontend's `parseErrorBody`/`ApiError` class correctly consumes this shape, and every audited page uses a shared, per-kind (`forbidden` vs. `offline` vs. generic) error-state renderer rather than one generic message.

---

9. Database & Persistence Audit

Status: VERIFIED for core entities; UI-ONLY for several settings categories

Real SQLite (`infra/data/orchestrator.sqlite3`, WAL mode, confirmed present on disk with `-shm`/`-wal` files from real usage), 7 migrations, restart-recoverable (`main.rs`'s `recover_incomplete_runs` on every boot).

Data	Persisted?	Evidence
Projects	**VERIFIED**	`0004_projects.sql`; `store.rs::insert_project/get_project/list_projects/update_project`
Users / API keys	**VERIFIED**	`0003_...sql` `api_keys` (hash-only column); no plaintext column exists anywhere
Roles	**VERIFIED**	`api_keys.role`, checked via `Role::parse`
AI settings/model config	**VERIFIED**	`0006_ai_settings.sql` singleton row; loaded at boot, written on every `PUT /ai/settings`, actually consulted by the router on every generation call
Generated/saved test cases	**VERIFIED**	`0005_test_cases.sql`; real `INSERT`/`SELECT` round-trip, test-proven
Execution runs/tasks/events	**VERIFIED**	`0001_core_execution.sql`
Evidence	**VERIFIED**	`0002_...sql` row + real local-disk file
Findings / bug drafts	**VERIFIED**	`0002_...sql` `bug_drafts`/`failure_clusters`
Audit log	**VERIFIED**	`0003_...sql` `audit_log`, written by every mutating handler traced + auth failures
Settings: General/SSO/Execution/Environments/Policies/Notifications	**UI ONLY (fake)**	`AIQAContext.tsx` `defaultSettings` — in-memory only, no backend table, no fetch/save call; own code comment admits "fabricated content"
Skill selections	**UI ONLY (by design, disclosed)**	`useSkillSelection.ts` own comment: "session/UI state only... never sent to or persisted by the backend"
Custom skills	**UI ONLY (by design, disclosed)**	`SkillManagerModal.tsx` tells the user directly: "added for this session only... will be gone on reload"
Generation sessions	**UI ONLY, undisclosed**	`AIQAContext.tsx` plain `useState`, pre-seeded with fake data (see AUD-006)

---

10. Authentication Audit

Status: VERIFIED

Every route except `GET /health` and the two WebSocket upgrade routes sits behind `require_auth` (`auth.rs:99-135`), which hashes the incoming bearer token (SHA-256) and looks it up (`find_api_key_by_hash`, excludes revoked keys). The two WebSocket routes authenticate via a `?key=` query parameter (the only mechanism browsers support for WS handshakes) through the identical hash-lookup path, additionally checking project access before allowing a subscription — confirmed via 9 passing `ws_tests::be14_*` tests including `missing_key_is_rejected` and `cross_project_subscription_is_rejected`.

Live-session confirmation (this audit, not a prior report): the bootstrap admin key was minted fresh, an unauthenticated `curl GET /health` returned `200 ok`, and an authenticated request through the real browser UI was confirmed with the in-app message "This key was accepted by a real, protected backend request."

Key management: `POST/GET/DELETE /admin/api-keys` are Admin-role-gated; keys are stored hash-only (`api_keys.key_hash`, no plaintext column exists); `list_api_keys` never selects the hash column, so it cannot leak even to an admin's own list view.

---

11. RBAC & Multi-Tenancy Audit

Status: VERIFIED

Three ordered roles (`Viewer < Operator < Admin`). Concrete `require_role` gates confirmed at file:line for project creation (Admin), project update, run creation/approve/reject/cancel, all test-case mutations, API-key management (Admin), and finding review (Operator). Read-only GETs correctly have no role gate.

Multi-tenancy: `require_project_access` is enforced at the handler layer for ~50 routes via `require_scoped_project`/`require_resource_in_project`, and filtering is real SQL (`WHERE project_id = ?`) at the store layer, not app-level post-filtering. This audit independently re-ran 12 named cross-tenant isolation tests live (`actual_be9_cross_project_isolation_is_enforced_on_every_scoped_route`, `be12_dashboard_and_kpis_do_not_leak_across_projects`, `b14_bug_drafts_are_project_scoped`, etc.) — **all passed**.

One historical IDOR (`review_bug_draft` previously had no project check at all) is confirmed **fixed** in current code, with the fix's own inline comment describing the bug it closed — a good sign of a team that documents its own past mistakes rather than erasing them.

One low-severity, by-design exception: `GET /metrics` returns platform-wide aggregate counts (not row-level data) to any authenticated role regardless of project scope (AUD-021).

---

12. Test Case Generator Audit

Status: STUB-FAKE-PLACEHOLDER on the only path a real user exercises; VERIFIED (unreached) in the backend

This is the audit's central finding, detailed fully in §13. Summary of the specific sub-questions from the brief:

Question	Answer
Is `selectedSkillIds` sent to the backend?	**No.** Real UI state, never transmitted (`useSkillSelection.ts` says so itself).
Do skills alter generation behavior?	**No, in the shipped UI** (output depends only on `selectedCategories`, a different taxonomy). **Yes, in isolated backend code** (test-proven), never invoked.
Do custom skills alter behavior?	Same as above — real backend support exists (falls back to the caller's own description verbatim), never reached.
Is AI genuinely invoked?	**No**, in the live flow. A local `setTimeout` template pipeline runs instead, every time.
Is AI response schema-validated?	**Yes**, in the real, unreached backend path (`serde_json` typed deserialization, real failure on malformed output).
Is invalid AI output handled?	**Yes** (unreached) — never silently faked as success.
Are duplicates handled?	**Yes** (unreached) — real normalized-title exact-match dedup.
Is traceability real?	**No, in the live UI** (fabricated string interpolation from filename/slot only — document content is never read). **Yes** (unreached) in the backend, which re-verifies the model's claimed source text as a literal substring before trusting it.
Are unsupported requirements fabricated?	The live UI fabricates plausible-sounding traceability text when no real requirement was extracted — see AUD-004/AUD-006.
Is origin labeling truthful?	**Yes** — the local generator is honestly labeled "LOCAL TEMPLATE GENERATION" with no false AI claim.

---

13. AI Generation Audit

Status: BACKEND ONLY (real and tested) / UI ONLY (fake, disconnected)

The chain breaks at the very first hop. `TestCaseGeneratorPage.tsx`'s `handleTriggerGenerate` (lines 422–707) is a pure client-side `setTimeout` chain assembling **hardcoded per-category template strings** — verified identical output for the same category on repeated runs, e.g. "OAuth 2.0 PKCE Code Verifier Hash Validation" every time for a Security-category request. The page's own comments disclose this ("this local, fixed-template pipeline does not vary its per-category output by selected skill... selection is real, its effect on local generation is not yet implemented"), and the UI shows a static "LOCAL TEMPLATE GENERATION" badge plus an `aiAnalysis` string reading "Locally assembled from a fixed category template — no AI model was called."

Meanwhile, the real pipeline is genuinely built and tested:

- `OllamaExecutor::execute` makes a real `POST {base_url}/api/generate` with `format: "json"`.
- `ai_generation.rs::generate_test_cases` calls the central router end to end, validates structured output via serde, enforces real traceability (re-checks the model's claimed `source_reference` as a literal substring of the real requirement text, stripping anything unverified), and deduplicates by normalized title.
- Integration tests exist that **panic if a live Ollama call fails** (`intelligence.rs::live_ollama_different_failure_clusters_produce_different_real_hypotheses`) — proof this is a real, non-mocked path when Ollama is actually running.
- Live-session evidence: Ollama at `:11434` was **not reachable** in this dev environment (confirmed via `curl` failure and via the orchestrator's own real startup log: *"Ollama discovery: not reachable"*), and no cloud-provider API keys were set — so even a direct call to the real endpoint would presently and honestly return `generation_method: "fallback"` with an empty result set, never a fabricated success.

Net assessment: this is not a case of AI generation being fake. It is a case of a real, correctly-engineered AI generation system having zero wiring to the product surface that claims to offer it. This is architecturally the easiest kind of gap to close (see §41, AUD-001).

---

14. Skills Architecture Audit

Status: BACKEND ONLY (real, tested, unreached)

`ai_generation.rs::builtin_instruction` maps each of 8 skill names (Functional, Edge Case, Security, Performance, API, Database, RBAC, Business Case Testing) to a distinct instruction block; `build_prompt` concatenates one block per selected skill; a unit test asserts a Functional-only prompt and a Security-only prompt differ, and that "injection" appears only in the security prompt. This is real, differentiated, evidence-based behavior — entirely inside code that the shipped Test Case Generator never calls (§13).

---

15. Custom Skills Audit

Status: STUB-FAKE-PLACEHOLDER (explicitly disclosed)

Created via `SkillManagerModal.tsx`, stored in a plain in-memory `useState` array with an in-page monotonic counter — never written to any store. The UI is directly honest about this: *"added for this session only — not saved anywhere, and will be gone on reload."* The backend would honor a custom skill if one were ever sent (falls back to the caller's own description verbatim, test-proven), but per §13, nothing ever sends it.

---

16. MCP Audit

Status: NOT CONFIGURED / NOT IMPLEMENTED

Stated plainly, per the audit brief's own instruction not to round this up to "partial." Zero MCP-related code exists anywhere in `services/orchestrator/src` (grepped, zero hits). `McpConnectModal.tsx` collects no input fields, stores nothing, and its own doc comment states outright that a fresh inspection confirmed no MCP server configuration, connector backend, credential storage, or plugin registry exists anywhere in the repository. `MCP_BACKEND_CONFIGURED = false as const` is a disclosed, hardcoded constant.

---

17. AI Provider Discovery Audit

Status: PARTIAL

Provider	Discovery mechanism	Real?
Ollama	Real `GET {base_url}/api/tags`, real timeout (default 3s), distinguishes connection failure / bad response / success	**VERIFIED real**
Gemini	`std::env::var("GEMINI_API_KEY")` non-empty check only	**STUB** — no live call ever made
OpenAI	Same pattern, `OPENAI_API_KEY`	**STUB**
Anthropic	Same pattern, `ANTHROPIC_API_KEY`	**STUB**
`OpenAiCompatible` / `Custom` enum variants	Declared, never constructed anywhere (0 grep hits)	**Dead code**, not backed by anything

The startup log line quoted verbatim from this session's real run — *"AI infrastructure discovery complete: 4 provider(s) checked, 0 healthy"* — is the real, honest output of exactly this mechanism: 4 providers checked (Ollama + 3 stubs), 0 healthy because Ollama wasn't reachable and no cloud keys were set.

---

18. AI Model Discovery Audit

Status: PARTIAL

Real for Ollama only — model `id`, `family`, `parameter_size` are read verbatim from the real `/api/tags` response; `context_window` is honestly left `None` with a comment explaining `/api/tags` doesn't carry it. The three cloud providers never populate a model list under any configuration (`models: Vec::new()` is unconditional) — there is no `/v1/models`-equivalent call anywhere in the codebase for any of them.

---

19. AI Health Check Audit

Status: VERIFIED for what is checked; PARTIAL overall

Ollama health is genuinely derived from the real HTTP round-trip (`Unavailable` on connection error, `Error` on bad status, `Healthy` only on success with >=1 model), bounded by a real 60-second TTL cache (`ensure_fresh`), not re-checked on every render and not a fixed value. Cloud-provider "health" is, and is honestly labeled as, nothing more than "is an environment variable non-empty" — it can report `Available`/`NotConfigured` but structurally can never report `Healthy`.

---

20. AI Capability Matching Audit

Status: VERIFIED (structure and disqualification logic), PARTIAL (some fields are flat defaults)

`required_capabilities(task)` is a real, fixed per-task-type capability list. `capabilities_from_ollama` derives `text_generation`/`structured_output`/`vision`/`embedding`/`tool_use` from Ollama's real `capabilities` array — and genuinely **disqualifies** a model lacking a required capability (test-proven: a text-only model is never routed a vision task). `reasoning` is honestly flagged in-code as a flat `Basic` assumption for every model (not independently benchmarked), and `long_context` is always `Unknown` — real limitations, not concealed ones.

---

21. AI Model Router Audit

Status: VERIFIED

`AiInfrastructure::route_among` is a pure, network-free, independently testable scoring/selection function. It genuinely iterates every discovered provider×model pair and keeps the highest-scoring valid candidate — proven by an outcome-based test (`routing_prefers_strong_structured_output_over_weak_for_test_case_generation`) that gives the router two real, distinct models and confirms it picks the one with stronger structured-output capability, not a hardcoded answer. The same function backs both real generation and the Settings page's `routing_preview` — there is no separate "looks good" fake preview.

---

22. AUTO/MANUAL Routing Audit

Status: VERIFIED

Both modes are real, backend-enforced branches, not frontend-only labels. MANUAL requires both `preferred_provider` and `preferred_model` to be set and returns a real, typed error (`ProviderUnavailable`/`ModelUnavailable`) if either doesn't currently exist — test-proven, including the case where a previously-valid manual selection later disappears. AUTO scores every candidate, honoring a preferred-provider/model as a scoring bonus rather than a hard override unless it's the only qualifying candidate. `AiInfrastructure::generate()` reads `settings.mode` on every real call — the backend genuinely branches on it.

---

23. AI Settings Panel Audit

Status: VERIFIED end-to-end — the single most completely wired AI-related feature in the product

Full path independently confirmed real at every hop: UI (`SettingsPage.tsx`'s `AiIntelligencePanel`) -> dedicated save action (genuinely separate from the page's unrelated fake "Save Settings" button, §31 AUD-007) -> `aiqaApi.ai.*` typed client -> RBAC-gated (`Role::Operator`) Rust routes -> real SQLite `UPSERT` into the `ai_settings` singleton row (`0006_ai_settings.sql`) -> in-memory router state updated in the same request -> **the router actually reads this state on every subsequent real generation call**, and the setting survives a process restart (loaded at boot in `main.rs` before the server starts serving).

The one caveat: this is a fully real setting with, at present, no frontend-triggered generation call to apply it to (§13).

---

24. AI Fallback Audit

Status: VERIFIED

Fallback triggers only when `mode == Auto && fallback_enabled` — both MANUAL mode and a disabled-fallback setting correctly skip retry and report a **truthful failure**, test-proven (`generate_reports_truthful_failure_when_fallback_is_disabled`). When triggered, candidates are re-scored excluding the failed one and tried in score order; `fallback_used: true` is set truthfully only when a different model actually produced the result. If nothing qualifies at all, the outcome is `Failed` — never a fabricated success. At the API layer, `generation_method: "ai"|"fallback"` is always present and honest; `provider`/`model` fields only appear alongside `"ai"`.

---

25. Test Execution Hub Audit

Status: PARTIAL — real end-to-end for manually-created tasks; the AI-generation handoff path always simulates

Two distinct routes exist:

- **Path A (real):** a task created directly in the Execution Hub with a real `executor` field is genuinely dispatched — `ExecutorRegistry::resolve` -> real `Command`-based subprocess (`npx playwright test ...`, `npx newman run ...`, `python -m pytest ...`, `maestro test ...`, `schemathesis run ...`) -> real combined stdout/stderr capture -> real evidence file on disk + DB row -> real retrieval endpoint. This entire chain is genuinely real, sanitized (every param passes `sanitize_token`, no bypass found), and SSRF-guarded (real DNS-resolution IP-range check, including the literal cloud-metadata `169.254.169.254` case in tests).
- **Path B (simulated, disclosed):** handing off a *generated* test case to execution (`TestCaseGeneratorPage.tsx::handleHandoffReal`) **always** sends `{ simulate: {...} }`, never a real `executor` field — the UI states this outright: "No real automation script exists for a generated case yet — its execution outcome is an explicit, disclosed simulated result, not a genuine verification of the steps above."

`SimulatedExecutor` itself is a real, documented, non-default fallback (runs only when `task.executor` is `None` or an unrecognized string) — not a mislabeled real-tool impersonation. One real gap: an unrecognized/misspelled `executor` string on the legacy `POST /runs` endpoint silently falls through to `SimulatedExecutor` with no error (AUD-014) — reachable only via a direct API call, not through the shipped UI.

---

26. Testing Modules Audit

Status: mostly NOT IMPLEMENTED / UI ONLY as distinct capabilities

Discipline	UI?	Backend?	Real distinct executor?	Verdict
Functional / Edge Case / Security / API / Database / RBAC / Business-rule / Performance	Only as an AI-generation "skill" label (§14)	Only as prompt-flavor text	No	**NOT a distinct testing capability** — affects only generated *text*, never execution, and is itself unreached (§13)
Non-functional, Load, Stress, Spike, Volume	No dedicated page anywhere in `routeConfig.tsx` (14 routes checked)	No concurrency/ramp-up/duration/threshold code found anywhere (`tokio::spawn` fan-out, RPS, throughput — 0 hits)	No	**NOT IMPLEMENTED**, stated bluntly
RBAC Testing (page)	Static `EmptyState` redirect to `/audit`, no boundary tests performed	Real enforcement exists (`auth.rs`) but isn't exercised *by this page*	No	**UI ONLY** (the page); real enforcement lives elsewhere and is separately verified (§11)
Mobile / API / Custom / Feature-Map (pages)	Honest `EmptyState`/"not implemented" badges, self-documented as having no backing entity	—	No	**NOT IMPLEMENTED**, honestly disclosed
API testing (as a capability, not the page)	—	Real, via choosing `pytest`/`newman`/`schemathesis` as an Execution Hub task's `executor`	Yes	**VERIFIED** — but note: `schemathesis` isn't actually selectable in the Execution Hub's own dropdown (only playwright/newman/pytest/maestro are), a real UI/backend mismatch
Mobile (Maestro)	Execution Hub dropdown offers it, gated by approval policy	Real subprocess dispatch	Partially — real code, unverifiable result (no device, and this dev machine's `maestro` on PATH is a different, unrelated tool)	**PARTIAL**

Bottom line, stated as bluntly as the brief requests: there is no backend mechanic anywhere in this codebase that generates concurrent load, ramps traffic, runs for a duration, or asserts a threshold. Clicking "run" on anything performance-flavored produces, at best, a single-shot real-tool run or a `SimulatedExecutor` result — indistinguishable from any other task type.

---

27. Dashboard Truthfulness Audit

Status: VERIFIED — real, SQL-derived; one honestly-empty gap

Every widget on `DashboardPage.tsx` was traced to a real backend computation: total test count (`COUNT(*)` on `quality_nodes`), run/task state counts (`GROUP BY state`), execution reliability (`PASSED/(PASSED+FAILED+TIMEOUT)`, `None` until any terminal outcome exists — never a fake `0%`), average run duration (`AVG(julianday diff)`, nullable). The donut chart is a pure client-side reshape of already-real counts with an explicit empty state. No `Math.random()`, no hardcoded numeric placeholder, anywhere in the dashboard render path.

One real, honestly-handled gap: the "Execution Activity" timeline chart has no backend time-bucketed history endpoint; the frontend hardcodes `timelineData={[]}` at the call site and the component correctly renders its own "No Execution Activity Recorded" empty state rather than fabricating a trend line (AUD-028).

---

28. Reporting Audit

Status: VERIFIED

`ReportsPage.tsx` (the Findings & Quality Intelligence / export page) pulls real `Finding[]` from a real endpoint, fetches fresh Dashboard/KPI data at export time, and computes a client-side reconciliation from those two real datasets for a "Final Audit Report" view — no third fabricated fetch. Export mechanisms are real: `jspdf`/`jspdf-autotable`/`xlsx` are dynamically imported and build genuine downloadable files from already-fetched data. The Test Intelligence section pulls real, deterministic (non-LLM, by design) risk/failure/value/flakiness scores computed after every run, with nullable fields honestly left `null` rather than fabricated as `0%`.

---

29. Evidence/Audit Logging Audit

Status: VERIFIED

Evidence: real combined stdout/stderr capture from actual subprocess invocations, written to real files under `infra/data/evidence/`, indexed in SQLite, served via a real, path-traversal-guarded `GET /evidence/:id/content` endpoint that returns real bytes. Disclosed scope limit: only log-type evidence is captured today (no screenshots/video/trace), honestly reflected in the evidence-kind data rather than faked.

Audit log: a real `audit_log` table, written by every mutating handler traced (project/run/test-case/API-key/finding actions) plus every authentication failure, retrieved via a real, RBAC-gated, project-scoped endpoint, rendered with real loaded/empty/error states — no static or fabricated rows found.

Telemetry/metrics: `GET /metrics` returns genuine Prometheus-format text from six real SQL `COUNT`/aggregate queries. The frontend's own UI **explicitly discloses**, in an on-page warning, that no latency/p95/p99 data exists — an honest limitation, not a concealed one.

---

30. Security Audit

Status: no critical/high application-layer code vulnerability found; one high-severity unfixed dependency; several low/informational notes

ID	Finding	Severity	Evidence
AUD-013-sec	Legacy `POST /runs` silently falls back to `SimulatedExecutor` (fabricated PASS) for an unrecognized `executor` string	**Medium**	`executor.rs` `ExecutorRegistry::resolve`; only reachable via direct API call, not the shipped UI, which always sends a valid executor or an explicit `simulate` block
AUD-016	`xlsx` (production dependency, used in real Excel export) has 2 unfixed High-severity advisories: prototype pollution (CVSS 7.8) and ReDoS (CVSS 7.5), **no fix available**	**High**	`npm audit` (this session, live), `GHSA-4r6h-8v6p-xvw6` / `GHSA-5pgg-2g8v-p4x9`
AUD-017	`vitest` dev-dependency has a **Critical** CVE (CVSS 9.8, arbitrary file read/execute via its UI server)	**Critical CVSS, Low real risk**	`npm audit` (live); dev-only, not shipped in `dist/`, exploitable only if a developer runs `vitest --ui` locally
AUD-018	AI generation prompt concatenates user-supplied `requirement_text` with no escaping of embedded prompt-injection attempts	**Low**	`ai_generation.rs`; blast radius limited to the requester's own generation request — no cross-user path found
AUD-019	Real-looking Firebase web API key committed in the abandoned `apps/web/firebase-applet-config.json`	**Low**	Firebase web API keys are not bearer secrets (security is enforced by Firebase Security Rules), but real key material identifying a real GCP project should not be committed
AUD-020	Weak hardcoded dev credentials in `infra/docker-compose.yml` (`aiqa`/`aiqa12345`)	**Informational**	File's own comment confirms this stack isn't wired in / running
AUD-021	`/metrics` exposes platform-wide aggregate counts to any authenticated role regardless of project scope	**Low**	By design per code comment; no row-level tenant data exposed
AUD-022	WebSocket auth via `?key=` query parameter can land the raw key in proxy/access logs or browser history	**Low**	Standard, commonly-accepted workaround for a real browser limitation (WS can't set custom headers)
— (informational)	API key stored in browser `localStorage`, not an httpOnly cookie	**Informational**	Standard SPA bearer-token pattern; no XSS vector found in this pass, but structurally exposed if one existed

Verified secure (positive findings, not gaps): SQL injection — parameterized throughout, no `format!()`-built query found anywhere in `store.rs`; command injection — `sanitize_token`/`safe_param` allowlist applied to every parameter on every real executor, no bypass found, test-proven against shell metacharacters and path traversal; SSRF — real DNS-resolution-based IP-range guard including the literal cloud-metadata-IP test case; path traversal — evidence store canonicalizes and checks `starts_with(root)`; secrets — API keys are SHA-256-hashed, never stored or logged in plaintext outside the one documented one-time bootstrap print; error disclosure — internal errors are logged server-side only, clients always get a fixed generic message; CORS — an explicit origin allow-list, not a wildcard (the code's own comment documents this as a prior "B12" remediation).

31. Fake/Stub/Placeholder Audit

Status: mostly disclosed/legitimate; a handful of genuinely undisclosed Production Fake Behavior findings

Production Fake Behavior (undisclosed to the end user — the most serious tier)

- **AUD-005 — Fabricated "Connected" network status.** `ApplicationSourceSection.tsx::handleConnect` makes **zero network requests**. After a fixed 600ms delay it unconditionally reports `CONNECTED`, `httpStatus: 200`, `sslValid: true`, and a `Math.random()`-based fake ping time, for *any* syntactically valid URL — including ones that don't exist. The code's own comment ("Real ping/validation response simulation") is self-contradictory.
- **AUD-006 — Pre-seeded fake "AI-generated" test cases with no disclosure.** `AIQAContext.tsx`'s `activeGenerationSession` defaults to a hardcoded session (`gen-4401`) containing 3 fully-formed security test cases, shown immediately on first load — unlike a real generation, these carry **no** `aiAnalysis` disclosure and no `template-generated` tag, so nothing distinguishes them from a genuine result to a first-time user.
- **AUD-004 — Fake file-upload progress.** `DocumentUploadSection.tsx::handleFileUpload` never reads or transmits the file's actual bytes (no `FileReader`, no `fetch`) — it drives a fake `setInterval` progress bar to 100% and marks the document "READY" regardless of content.
- **AUD-007 — Settings page silently fake-persists most of its tabs.** Every tab except "AI Models" (General org info, Authentication/SSO toggle, Execution runner pool, Environments, Policies, Notifications) writes only to in-memory React state. Clicking "Save Settings" shows a green "Configuration Saved" checkmark **indistinguishable from a real save** — but nothing is sent to any backend, and the state is lost on refresh. The code is self-aware in a comment ("pre-existing fabricated content... left exactly as-is"), but this awareness is not surfaced to the user.

Suspicious (lower severity, worth fixing but low real-world impact)

- **AUD-008** — "Control Plane Daemon API Token" card is fully decorative static text; its "Rotate Token" button has no click handler at all, sitting directly beside the real, working "Replace Key" button.
- **AUD-009** — Environment switcher (Staging/Production/UAT/"Ephemeral PR-1482") is pure decoration; the selected value is read nowhere else in the app and affects zero API calls.

Legitimate (verified, no action needed)

`SimulatedExecutor` (documented, non-default, never mislabeled as a real tool); the 5+ explicitly-labeled "not implemented" stub pages; the local-template Test Case Generator itself (fake in effect, but honestly labeled — the deception risk is the *absence of a real alternative in the UI*, not a lie about what's shown); dashboard/AI-provider/reports code (all real, already covered above); ~15 legitimate HTML `placeholder=` attributes; numerous code comments documenting already-completed prior fake-data removals, spot-checked and confirmed accurate.

---

32. Test Quality Audit

Status: VERIFIED — genuinely high quality, not just high count

371 frontend tests (vitest) and 206 backend tests (cargo), both suites passing in full live runs during this audit. Critically, this audit independently confirmed these are **outcome-assertion tests, not render-smoke tests**: named backend tests directly assert cross-tenant isolation is enforced (not just "the route exists"), that RBAC denial actually returns 403 for a viewer role, that a routing decision genuinely prefers a stronger-capability model given two real candidates, and that a fallback failure is reported truthfully rather than silently. Frontend tests similarly assert real behavior, e.g. "SettingsPage — AI Intelligence panel... switching to MANUAL mode and saving sends a real PUT /ai/settings request with the chosen provider/model" and "shows a truthful Degraded status... never a fake Ready." This is a materially higher bar than the "component renders" anti-pattern the audit brief warns against, and this audit did not find that anti-pattern present in the suites reviewed.

---

33. Runtime Verification

Performed, not fabricated, this session:

- Application startup: **VERIFIED** — orchestrator built and ran on `:8080`, real bootstrap admin key minted; frontend dependencies installed and Vite dev server ran on `:3000`.
- Login/authentication: **VERIFIED** — bootstrap key entered through the real Settings UI, backend genuinely validated it ("This key was accepted by a real, protected backend request").
- Project selection / dashboard reachability: **VERIFIED** — `GET /api/v1/projects` returned real `200 OK` through the Vite proxy; Dashboard correctly showed "No project selected" (honest empty state, zero projects exist yet in this fresh DB — not a bug).
- Test Case Generator, Skills, AI routing/generation live click-through: **NOT MANUALLY VERIFIED** in-browser this session (evidence for these areas comes from code-level tracing + the existing passing test suites, which this audit judges sufficient given the depth of the test coverage found — but no fresh manual click-through screenshot of the Generator was captured).
- Execution / reports live click-through: **NOT MANUALLY VERIFIED** in-browser this session, for the same reason.

Where runtime verification was not performed, this is stated explicitly above rather than assumed from code alone.

---

34. Build & Typecheck Results

Check	Command	Result
Backend build	`cargo build` (via `cargo test`)	**PASS**
Backend tests	`cargo test`	**PASS — 206 passed, 0 failed, 2 ignored**
Frontend typecheck	`npm run lint` (`tsc --noEmit`)	**PASS — 0 errors**
Frontend tests	`npm test` (`vitest run`)	**PASS — 20 files, 371 tests, 0 failures**
Frontend production build	`npm run build` (`vite build`)	**PASS** — 2,967 modules; 1 warning (1.07 MB main chunk, no code-splitting)
`apps/web` typecheck	`npm run typecheck`	**PASS — 0 errors** (duplicate/abandoned app, checked for comparison only)
`apps/web` build	`npm run build`	**PASS** — 2,299 modules
Dependency audit (`forentend`)	`npm audit`	**6 vulnerabilities**: 1 critical (dev-only, `vitest`), 2 high (1 dev-only `vite`, 1 **production**, `xlsx`, no fix available), 3 moderate (dev-only)
Lint (dedicated linter)	—	**NOT APPLICABLE** — no ESLint/Clippy config found; `tsc --noEmit` is the project's own stand-in "lint" script
CI pipeline	—	**NOT CONFIGURED** — no `.github/` directory exists

---

35. Regression Results

No regressions were introduced by this audit (read-only, plus one already-reverted `npm install`/build artifact set). All pre-existing suites pass at their current counts; no test was skipped, deleted, or weakened to produce a pass. The 2 `#[ignore]`d backend tests are pre-existing, correctly gated on live-Ollama availability, and were not run (Ollama is not installed in this environment) — this is a `NOT APPLICABLE` result for those 2 tests, not a silent skip mislabeled as a pass.

---

36. Architecture Strengths

1. **Unusually rigorous backend test culture** — 206 passing tests that assert real outcomes (isolation, RBAC, truthful fallback), not smoke tests, cross-verified live in this audit.

2. **A codebase that documents its own past mistakes** — multiple modules contain comments explicitly describing a bug that was found and fixed (the bug-draft IDOR, the CORS wildcard, the structured-error-envelope gaps), which is a strong signal for maintainability and honesty.

3. **Real, working security fundamentals**: parameterized SQL throughout, a tested command-injection allowlist, a tested SSRF guard with real DNS resolution, hashed API keys, an explicit CORS allow-list.

4. **A genuinely well-designed AI router** (capability-based disqualifying scoring, tested AUTO/MANUAL branching, truthful fallback) that is centralized in one module rather than duplicated across the codebase.

5. **Consistent self-disclosure culture in the frontend** — most fake/incomplete features carry an in-app or in-code admission (badges, disclaimers, `EmptyState` pages) rather than silent fabrication.

6. **Real crash recovery and durable state** — WAL-mode SQLite, transactional inserts, a genuine startup recovery pass for runs left non-terminal by an unclean shutdown.

---

37. Architecture Weaknesses

1. **The single most valuable backend capability (real AI generation) has zero frontend wiring** — an architecture/integration gap, not a design flaw in either half.

2. **No performance/load/stress/spike/volume testing mechanic exists anywhere**, despite being a named product concept in the UI's skill/category taxonomy.

3. **The Settings page mixes one fully-real tab with several silently-fake ones behind an identical-looking success toast** — a genuine trust risk if a user relies on any tab other than "AI Models."

4. **No CI/CD pipeline** — every build/test/typecheck result in this report was obtained by manually running the command; nothing enforces these gates on new commits.

5. **No containerization** — `infra/docker-compose.yml` describes a target Postgres/Redis/MinIO architecture that isn't wired in or runnable in this environment; the real system relies on SQLite/local-disk plus a Windows-only Job Object containment mechanism (no equivalent exists for non-Windows hosts, and no executors are wired for any other platform per `main.rs`'s own comment).

6. **One production dependency (`xlsx`) has unfixed High-severity vulnerabilities with no upstream fix available.**

7. **Cloud AI providers (Gemini/OpenAI/Anthropic) are placeholders** — declared, env-checked, but never given a real client, so "multi-provider" is currently true only for one provider (Ollama).

8. **A duplicate, buildable-but-abandoned frontend (`apps/web/`) remains in the repo** with a committed real API key, inviting confusion about which frontend is authoritative.

---

38. Critical Gaps

Ranked by blast radius on the product's stated purpose:

1. **AI-powered test case generation does not work as shipped** (§13, AUD-001/002/003/004) — blocks the core value proposition.
2. **No performance/load/non-functional testing capability exists** (§26) — blocks a named, explicit scope area entirely.
3. **Settings page silently fake-persists most configuration** (§31, AUD-007) — a trust/correctness risk once discovered by a real operator.
4. **MCP is entirely absent** (§16) — disclosed, but a named scope area with zero implementation.
5. `xlsx` **production dependency has unfixed high-severity vulnerabilities** (AUD-016).

---

39. Full Findings Register

ID	Title	Module	Status	Severity	Fix Priority
AUD-001	Test Case Generator "Generate" button never calls the real, tested AI backend (`generate-ai` endpoint)	Frontend / AI Generation	STUB-FAKE-PLACEHOLDER (disclosed)	Critical	**P0**
AUD-002	`selectedSkillIds` captured in UI state, never transmitted to backend	Frontend / Skills	UI ONLY	High	**P0**
AUD-003	Custom Skills are session-only; explicitly lost on reload	Frontend / Skills	STUB (disclosed)	Medium	P1
AUD-004	Document upload never reads or transmits file content; fake progress bar	Frontend / Test Generator	STUB-FAKE-PLACEHOLDER (undisclosed)	High	P1
AUD-005	App-source "Connect" fabricates HTTP/SSL/ping status with zero network request	Frontend / Test Generator	Production Fake Behavior	High	P1
AUD-006	Pre-seeded fake "AI-generated" test cases shown on first load, no disclosure	Frontend / Test Generator	Production Fake Behavior	High	P1
AUD-007	Settings page silently fake-persists non-AI tabs behind a real-looking success toast	Frontend / Settings	Production Fake Behavior	Medium	P1
AUD-008	"Control Plane Daemon API Token" card is fully decorative; "Rotate Token" button has no handler	Frontend / Settings	Suspicious	Low	P2
AUD-009	Environment switcher is decorative, affects nothing	Frontend / Header	Suspicious	Low	P2
AUD-010	Generated-test-case handoff always simulates; never reaches a real executor	Frontend / Execution	PARTIAL (disclosed)	Medium	P1
AUD-011	MCP not implemented anywhere in the codebase	Backend+Frontend / MCP	NOT IMPLEMENTED (disclosed)	Informational	P3
AUD-012	No performance/load/stress/spike/volume testing mechanics exist	Backend / Testing	NOT IMPLEMENTED	High	P1

ID	Title	Module	Status	Severity	Fix Priority
AUD-013	"Testing module" pages (RBAC/Mobile/API/Custom/Feature Map) are non-functional stubs	Frontend / Testing pages	NOT IMPLEMENTED (disclosed)	Medium	P2
AUD-014	Legacy `POST /runs` silently falls back to `Simulated Executor` (fabricated PASS) for an unrecognized `executor` name	Backend / Execution / Security	Real gap (API-only, not UI-reachable)	Medium	P1
AUD-015	Gemini/OpenAI/Anthropic are env-var-presence stubs with no real client/executor	Backend / AI Router	STUB	Medium	P2
AUD-016	`xlsx` production dependency: unfixed High-severity prototype-pollution + ReDoS advisories	Frontend / Dependencies / Security	Real vulnerability, no fix available	High	P1
AUD-017	`vitest` dev dependency has a Critical CVE (arbitrary file read via UI server)	Frontend / Dependencies / Security	Real vulnerability, dev-only blast radius	Low (practical)	P2
AUD-018	AI generation prompt has no escaping against embedded prompt-injection text	Backend / AI Generation / Security	Real, low blast radius	Low	P3
AUD-019	Real-looking Firebase API key committed in abandoned `apps/web`	Frontend (dead code) / Security	Real exposure, low impact	Low	P3
AUD-020	Weak hardcoded dev credentials in `infra/docker-compose.yml`	Infra / Security	Real, unused stack	Informational	P3
AUD-021	`/metrics` exposes platform-wide aggregate counts regardless of project scope	Backend / Observability / Security	By-design minor leak	Low	P2
AUD-022	WebSocket auth via `?key=` query string can land raw keys in logs/history	Backend / Auth / Security	Standard workaround caveat	Low	P3
AUD-023	No CI/CD pipeline configured anywhere in the repo	DevOps	NOT CONFIGURED	Medium	P1
AUD-024	No containerization / Docker deployment path; Windows-only process containment	DevOps	NOT IMPLEMENTED	Medium	P2
AUD-025	Frontend production bundle has a 1.07 MB uncode-split main chunk	Frontend / Build	Quality warning	Low	P2

ID	Title	Module	Status	Severity	Fix Priority
AUD-026	README's "not yet implemented: LLM integration" claim is stale — real Ollama-backed generation exists (but is unreached, AUD-001)	Docs	Stale documentation	Informational	P3
AUD-027	Abandoned duplicate frontend (`apps/web/`) still builds cleanly, risks confusion about which app is authoritative	Repo hygiene	Dead code	Low	P3
AUD-028	Dashboard "Execution Activity" timeline chart has no backend support; hardcoded empty (honestly disclosed)	Frontend+Backend / Dashboard	NOT IMPLEMENTED (disclosed)	Low	P2/P3

---

40. Production Readiness Scorecard

Area	Score /10	Verdict
Frontend	7	Real, well-tested (371/371), clean build — but the flagship generator feature is a disconnected facade
Backend	9	Extremely well-tested (206/206), real persiste nce/auth/RBAC/policy/execution/evidence, thoughtful security hardening
AI Infrastructure	7	Genuinely real router/scoring/AUTO-MANUAL/fallback, tested — but only 1 of 4 declared providers (Ollama) actually works
Test Case Generation	3	Real backend exists and is well-tested; the shipped product feature is 100% disconnected local templates
Skills	4	Real, tested skill-to-prompt differentiation server-side; completely inert in the shipped UI
AI Model Routing	8	Real, tested scoring/selection/fallback, persisted settings — but its only reachable consumer is the Settings status panel
Settings	4	AI Models tab fully real; every other tab silently fake-persists behind a real-looking success toast
Test Execution	7	Manually-created runs are genuinely real end-to-end across 5 real tools; generated-case handoff never reaches them
Security	8	No critical/high application-layer vulnerability found; one unfixed high-severity production dependency; a few low/informational notes
RBAC	9	Extremely well-tested three-role model with real, enforced project isolation; one dedicated "RBAC Testing" page is a non-functional stub (doesn't affect real enforcement)
Persistence	8	Real, durable, restart-recoverable SQLite for all core entities; several Settings categories and all skill/session state are frontend-only
Reporting	8	Real Findings/Dashboard/KPI-backed reports with genuine client-side export
Observability	6	Real audit log and real Prometheus metrics; no latency data exists at all (disclosed), no execution-activity time series
Test Quality	8	577 total passing tests (371 FE + 206 BE) proven, by direct inspection, to assert real outcomes rather than render-smoke checks
Overall	**6.8**	**CONDITIONAL GO** — an unusually solid, honest, well-tested architecture whose single most important product feature is not wired to its own real, working backend

41. Recommended Fix Order

P0 — Critical / Blocks Real Usage

- **AUD-001 + AUD-002**: Add `aiqaApi.testCases.generateAi(projectId, {requirement_text, skills})` calling the real `POST .../test-cases/generate-ai` endpoint; wire `TestCaseGeneratorPage.tsx`'s Generate action to it instead of the local template pipeline; pass `selectedSkillIds` (resolved to `SkillInputBody[]`, including custom-skill descriptions) through. Render the real `generation_method`/`provider`/`model`/`fallback_reason` fields truthfully (the backend already produces them correctly). **This single change activates real AI generation, real skill differentiation, real enforced traceability, and real dedup — all of which already exist and are tested.**

Acceptance criteria: selecting different skills for the same requirement text produces demonstrably different generated output; a generated case's `generation_method` is visibly rendered; with no AI provider reachable, the UI shows an honest failure/fallback state rather than falling back to the old fake templates silently.

P1 — High Priority

- **AUD-004/005/006**: Replace the fabricated file-upload/app-source/pre-seeded-session behavior with either real implementations (read file content and send it; make a real reachability check; clear the seeded demo session on first real load or clearly label it as a sample) or, at minimum, the same honest in-app disclosure pattern already used elsewhere in this codebase.
- **AUD-007**: Either wire the remaining Settings tabs to real backend persistence, or change the "Configuration Saved" toast so it cannot be mistaken for a real save (e.g., a visible "local only, not saved to server" notice, matching this codebase's own established disclosure pattern).
- **AUD-012**: Decide product scope for performance/load/stress/spike/volume testing — either build a minimal real mechanism (even a simple concurrent-request generator with duration/threshold would materially change this from "0" to "something real") or remove/relabel these as explicitly out of scope, consistent with how Mobile/Custom/FeatureMap pages are already honestly labeled.
- **AUD-014**: On the legacy `POST /runs` path, reject (not silently simulate) an `executor` value that doesn't match a registered executor.
- **AUD-016**: Address the `xlsx` vulnerability — evaluate a maintained fork/alternative (e.g. `exceljs`) since no upstream fix exists for the installed package.
- **AUD-023**: Stand up a minimal CI pipeline running exactly the four commands this audit ran by hand (`cargo test`, `tsc --noEmit`, `vitest run`, `vite build`) on every push/PR.

P2 — Important

- AUD-003, AUD-010, AUD-013, AUD-015, AUD-017, AUD-021, AUD-024, AUD-025, AUD-028 — see individual entries in §39 for specifics.

P3 — Future

- AUD-011 (MCP — real scope decision needed, not a bug), AUD-018, AUD-019, AUD-020, AUD-022, AUD-026, AUD-027.

---

42. Next Implementation Phase

The single highest-leverage next unit of work is **AUD-001/002**: wiring the Test Case Generator UI to the real, already-built, already-tested `generate-ai` backend. Every other AI-related score in §40 (AI Infrastructure, Skills, AI Model Routing) is held down primarily by this one disconnection, not by any missing backend capability. This should be scoped as its own unit before any new AI feature work, since building more capability on top of an unwired foundation would only widen the gap between what the backend can do and what a user can reach.

---

43. Known Limitations of This Audit

- Runtime click-through verification (§33) covered startup, auth, and dashboard reachability but **not** a fresh, in-browser exercise of the Test Case Generator, Execution Hub, or Reports pages in this specific session — those conclusions rest on code tracing plus the existing (large, outcome-asserting) automated test suites, judged sufficient given their depth, but explicitly not the same as a fresh screenshot.
- `apps/web/` and `services/workers/` were confirmed-inactive/stub but not exhaustively audited line-by-line, since both are outside the live product surface.
- Dependency security scanning covered `npm audit` for `forentend`; no equivalent `cargo audit` was run against the Rust dependency tree (tool not confirmed available in this environment) — Rust dependency CVEs are therefore **UNVERIFIED**, not confirmed absent.
- No load/penetration testing against the live running orchestrator was performed beyond the specific SSRF/injection code-path checks described in §30.

---

44. Final Verdict

CONDITIONAL GO

Core architecture works — and works unusually well for its stage: real persistence, real multi-tenant RBAC enforced and proven under live test execution, a real and well-designed AI router, real test execution against five genuine external tools, and a security posture with no critical or high application-layer vulnerability found in the code itself. This is not a codebase full of fake behavior; the fakes found are, with a small number of clearly-identified exceptions (§31), honestly disclosed.

But clearly identified P0/P1 gaps must be resolved before this can be considered ready for real usage or a broader manual testing pass: most importantly, **the product's core AI-generation capability is not reachable from its own user interface**, despite being fully built and tested on the backend. A second-tier but real gap is the complete absence of any performance/load/stress testing mechanism, and a third is a Settings page that can silently mislead an operator about what is actually saved.

Final Question Answers

1. **Can AIQA currently generate real test cases using AI?** Not as shipped. The backend genuinely can (real, tested Ollama-backed pipeline — `ai_generation.rs` + `ai_router.rs`), but the only user-facing "Generate Test Cases" button never calls it (§13, AUD-001).

2. **Do selected Skills genuinely change generated test-case behavior?** No, in the shipped UI (output depends only on category selection, never on skills). Yes, provably, in the unreached backend code (test-proven prompt differentiation) (§14).

3. **Can AIQA automatically discover available AI providers/models?** Partially. Real, HTTP-based discovery with real model enumeration exists for Ollama only. Gemini/OpenAI/Anthropic are environment-variable-presence checks with no live reachability test and no model enumeration ever (§17–18).

4. **Does AUTO mode genuinely select a model based on task requirements?** Yes — a real, tested, capability-based scoring algorithm, backend-enforced (§21–22).

5. **Does MANUAL mode genuinely change the model used?** Yes — a real, tested, backend-enforced branch with correct error handling when the chosen model is unavailable (§22).

6. **Are AI provider/model statuses truthful?** Yes — no status is ever inflated; cloud providers correctly never claim "Healthy" without a live check, and credentials are test-proven to never leak into status responses (§19).

7. **Can AIQA execute real tests, or are any results simulated?** Both, and truthfully so for manually-created tasks: five real tool integrations exist and genuinely dispatch (Playwright/Newman/Pytest/Schemathesis fully verified; Maestro real-but-unverifiable-without-a-device). The one meaningful and disclosed exception: a *generated* test case's handoff to execution always simulates, never reaches a real tool (§25).

8. **Which major modules are genuinely production-ready?** The backend orchestrator core (execution engine, scheduler, evidence, auth/RBAC, persistence, policy, dashboard/KPI SQL, audit log, metrics) and the frontend's Dashboard/Reports/Evidence/Audit/Telemetry/AI-Settings/Execution-Hub(manual-path) pages. The AI router is production-ready in isolation.

9. **What blocks AIQA from reaching the user's core goal?** One integration gap (§13/§37/§38, item 1) — a real, tested AI generation backend has no frontend caller — is the largest single blocker. Secondary blockers: no performance/load testing capability at all, a Settings page that can mislead about what's actually saved, and MCP being entirely unimplemented.

10. **What is the exact next implementation priority?** Wire `TestCaseGeneratorPage.tsx`'s generation flow to the real `POST .../test-cases/generate-ai` endpoint, including `selectedSkillIds` transport and truthful `generation_method`/provider/model rendering (§42, AUD-001/002). This one unit of work activates the largest concentration of already-built, already-tested, currently-inert backend capability in the entire codebase.

End of report. Prepared as a fresh, independent, evidence-based audit per the request's methodology; no claim above rests on a prior `AIQA_*_REPORT.md` file.